

Graphisme dans le jeu vidéo

Table des matières

Session 1 : Géométrie et Découverte de l'API	3
Présentation du cours	3
Découverte de l'environnement : Babylon.js et le Playground	4
Rappels mathématiques : Vecteurs et Matrices	4
La géométrie bas niveau	5
Atelier pratique : Création d'un mesh à la main	6
Démonstration : Construire un maillage de A à Z	8
Session 2 : Transformations et Espaces de Coordonnées	11
Le voyage d'un Vertex : Le Pipeline de Transformation	11
Le « Pourquoi » de la Matrice 4x4	12
Hiérarchie de scène : Le Parenting	12
Atelier : Manipuler la Hiérarchie dans Babylon.js	13
Concepts clés à retenir	14
Session 3 : Création de Shaders dans Babylon.js (Partie 1)	15
Qu'est-ce qu'un Shader ?	15
Le Pipeline de Rendu Programmable	15
GLSL : Le langage des Shaders	15
Shaders dans Babylon.js : ShaderMaterial	15
Uniforms et Varyings	16
Session 4 : Création de Shaders dans Babylon.js (Partie 2)	17
Éclairage dans un Shader	17
Textures dans les Shaders	17
Normal Mapping	17
Effets visuels : Procedural Shaders	17
Node Material Editor	18
Session 5 : Création d'éclairage dans Babylon.js	19
Types de Lumières dans Babylon.js	19
Paramètres de Lumière	19
Modèles d'Éclairage	19
PBRMaterial dans Babylon.js	19
Multiples Lumières et Performance	20
Session 6 : Stratégies d'Ombres et Réflexions	21
Shadow Mapping	21
Ombres dans Babylon.js	21
Types de Shadow Maps	21
Réflexions	21
Réflexions dans Babylon.js	21
Image-Based Lighting (IBL)	22
Session 7 : Création de Post-Process dans Babylon.js	23
Qu'est-ce qu'un Post-Process ?	23
Post-Process Pipeline dans Babylon.js	23
Post-Process Intégrés	23

Post-Process Custom	23
HDR et Tone Mapping	24
Session 8 : Rendering Pipeline (Partie 1)	25
Vue d'Ensemble du Pipeline	25
Vertex Shader en Détail	25
Rasterization	25
Fragment Shader en Détail	25
Tests et Blending	26
Session 9 : Rendering Pipeline (Partie 2)	27
Forward vs Deferred Rendering	27
G-Buffer	27
Culling	27
Draw Calls et Batching	27
Babylon.js et le Pipeline	28
Session 10 : Techniques d'Optimisation et Profilage (Partie 1)	29
Pourquoi Optimiser ?	29
Outils de Profilage	29
Métriques Clés	29
Optimisation Géométrique	29
Optimisation des Draw Calls	30
Session 11 : Techniques d'Optimisation et Profilage (Partie 2)	31
Optimisation des Textures	31
Optimisation des Shaders	31
Level of Detail (LOD) dans Babylon.js	31
Gestion de la Mémoire GPU	31
WebGPU vs WebGL	32
Session 12 : Exploration d'une API de Programmation Graphique	33
APIs Graphiques : Vue d'Ensemble	33
WebGPU : La Nouvelle API Web	33
Comparaison WebGL vs WebGPU	33
WebGPU dans Babylon.js	33
WGSL : Le Langage de Shader WebGPU	34
Conclusion du Cours	34

Session 1 : Géométrie et Découverte de l'API

🔗 Objectifs de la session

Présenter le cours et son fonctionnement, découvrir l'écosystème Babylon.js, rappeler les notions mathématiques essentielles, et construire un maillage 3D à la main pour comprendre la structure intime d'un modèle.

Présentation du cours

Orientations et évaluation

Ce cours explore la programmation graphique 3D temps réel appliquée au jeu vidéo, en utilisant entre autres Babylon.js comme moteur, mais aussi d'autres outils et frameworks (Unity, Unreal Engine, Blender, Godot).

Évaluation :

- Travaux pratiques (TP) : Mise en pratique des concepts de chaque session.
- Projet final : Création d'un projet utilisant l'ensemble des principes abordés.

Approche pédagogique : Chaque session combine théorie et atelier pratique. Le code est écrit en JavaScript/TypeScript et s'exécute directement dans le navigateur. Des démonstrations seront aussi faites sur les engins commerciaux, pour montrer les analogies, la programmation graphique étant indépendante du moteur utilisé.

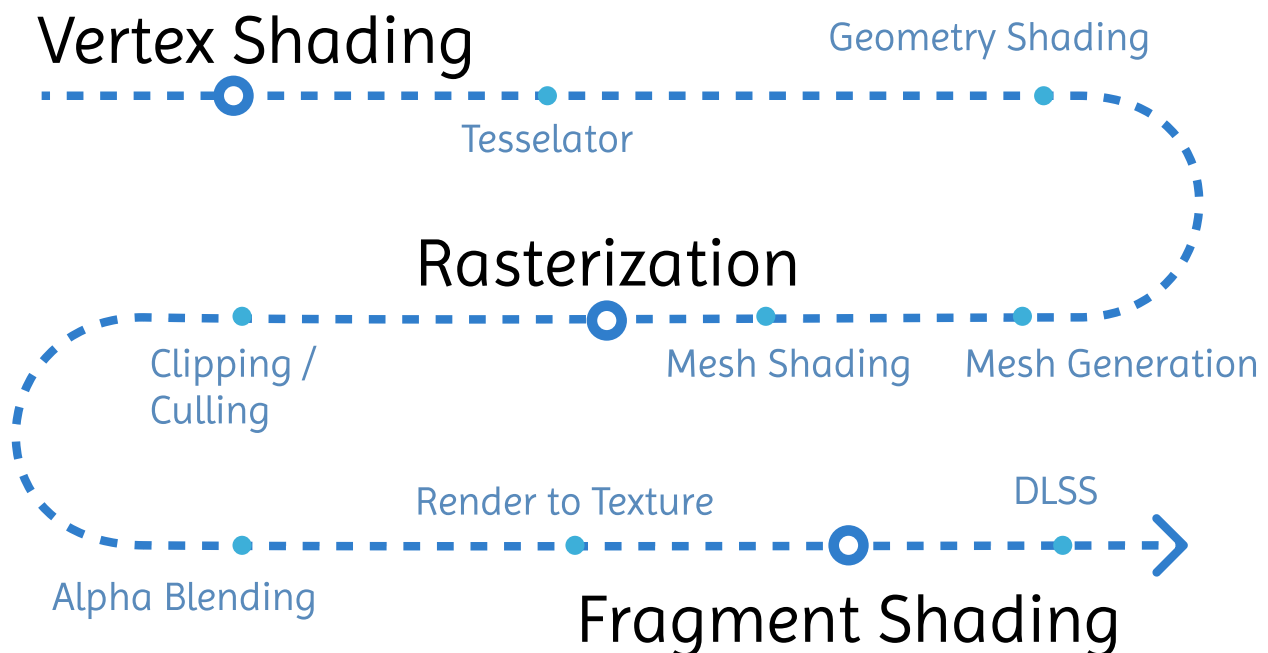


Fig. 1. – Notre parcours dans l'univers du GPU.

Introduction à la programmation graphique

La programmation graphique consiste à décrire au GPU (carte graphique) comment transformer des données mathématiques (points, vecteurs, matrices) en pixels colorés à l'écran. C'est la rencontre entre les mathématiques, la physique (optique) et l'informatique.

Contrairement à la programmation classique (CPU, séquentielle), la programmation graphique exploite le parallélisme massif du GPU : des milliers de calculs exécutés simultanément.

Découverte de l'environnement : Babylon.js et le Playground

L'écosystème Babylon.js

Babylon.js est un moteur 3D open-source écrit en TypeScript, s'exécutant dans le navigateur via WebGL et WebGPU. Il offre :

- Un moteur de rendu temps réel (PBR, lumières, ombres, post-process).
- Un éditeur visuel de shaders (Node Material Editor).
- Une API complète pour la création de scènes, d'animations et d'interactions.

Avantage clé : Aucune installation requise. Tout s'exécute dans le navigateur, et il est possible de partager facilement les exemples via URL.

🔗 Le Babylon.js Playground

Le **Playground** (<https://playground.babylonjs.com>) est l'outil idéal pour prototyper :

- Zéro installation : on écrit du code JS directement dans le navigateur.
- Itération rapide : modification en direct, rendu instantané.
- Partage : chaque scène a une URL unique partageable.
- Modèles : des centaines d'exemples officiels pour apprendre.

C'est l'environnement que nous utiliserons pour les ateliers pratiques.

Rappels mathématiques : Vecteurs et Matrices

Vecteurs (`Vector3`)

Un vecteur représente une direction et une magnitude dans l'espace 3D. Dans Babylon.js, on utilise la classe `BABYLON.Vector3`.

Opérations essentielles :

```
const a = new BABYLON.Vector3(1, 2, 3);
const b = new BABYLON.Vector3(4, 5, 6);

a.add(b);           // Addition
a.subtract(b);      // Soustraction
a.scale(2);         // Multiplication par scalaire
BABYLON.Vector3.Dot(a, b); // Produit scalaire
BABYLON.Vector3.Cross(a, b); // Produit vectoriel
a.normalize();      // Vecteur unitaire
a.length();         // Magnitude (norme)
```

- **Produit scalaire** : Mesure l'alignement entre deux vecteurs. Utilisé pour l'éclairage (angle entre normale et lumière).
- **Produit vectoriel** : Produit un vecteur perpendiculaire aux deux autres. Utilisé pour calculer les normales de surface.

Matrices (`Matrix`)

Une matrice 4x4 représente une **transformation** dans l'espace (translation, rotation, scale, ou combinaison). Babylon.js utilise `BABYLON.Matrix`.

Transformations courantes :

```
// Translation
const T = BABYLON.Matrix.Translation(1, 0, 0);

// Rotation (autour de Y)
const R = BABYLON.Matrix.RotationY(Math.PI / 4);

// Scale
const S = BABYLON.Matrix.Scaling(2, 2, 2);

// Composition : T * R * S (l'ordre compte !)
const world = T.multiply(R).multiply(S);
```

Pourquoi 4x4 ? La 4ème coordonnée (w) permet de unifier translations et rotations dans une seule matrice. Un point (x, y, z) devient $(x, y, z, 1)$ en coordonnées homogènes.

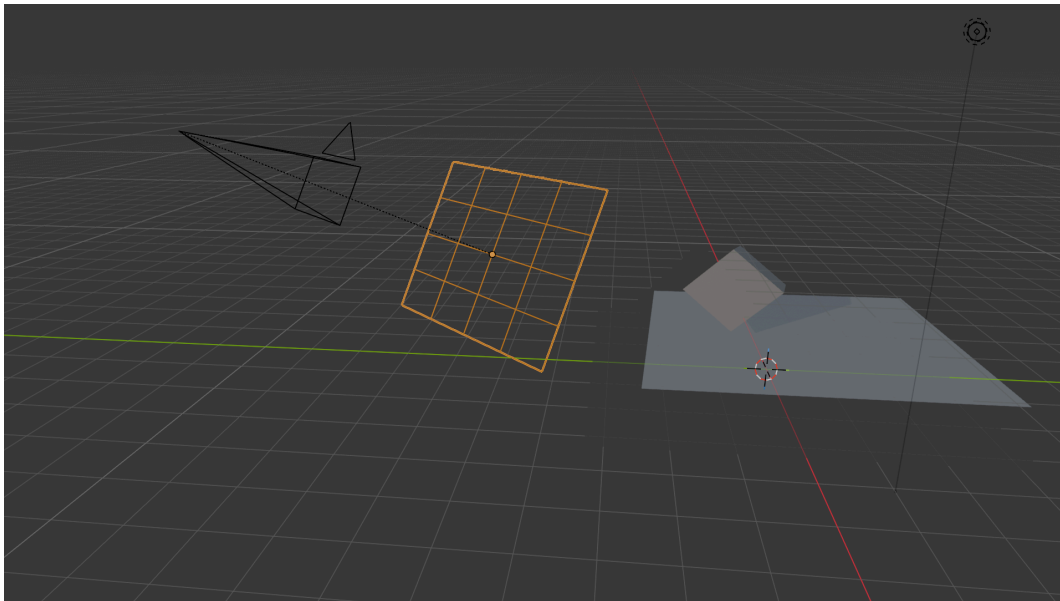


Fig. 2. – L'écran est une projection du monde en 3 dimensions.

La géométrie bas niveau

Composantes d'un Vertex

Un **vertex** (sommet) n'est pas qu'un point dans l'espace. Il peut contenir plusieurs attributs :

- **Position** : Coordonnées (x, y, z) dans l'espace objet. Obligatoire.
- **Normale** : Vecteur perpendiculaire à la surface, utilisé pour l'éclairage. Direction (n_x, n_y, n_z) .
- **UV** : Coordonnées de texture (u, v) dans $[0, 1]$, reliant le sommet à un pixel de la texture 2D.
- **Tangente** : Vecteur tangent à la surface, utilisé pour le normal mapping (espace tangent).
- **Couleur** : Couleur RGBA par sommet (r, g, b, a) , utilisée pour le vertex coloring.

Ces attributs sont stockés dans des **Vertex Buffers** séparés et envoyés au GPU.

Types de primitives

En graphisme temps réel, toute surface est décomposée en triangles. Les types de primitives GPU sont :

- **Points** : Un point par vertex (rarement utilisé).
- **Lignes** : Une ligne par paire de vertices.
- **Line Strip** : Lignes connectées en chaîne.
- **Triangles** : Un triangle par triplet de vertices. C'est le type dominant.
- **Triangle Strip** : Triangles connectés partageant des arêtes (plus économe en vertices).
- **Triangle Fan** : Triangles partageant un vertex central.

Le triangle est la primitive de base car :

- Trois points définissent toujours un plan (pas d'ambiguïté).
- L'interpolation des attributs (UV, normales) est bien définie à l'intérieur.

Atelier pratique : Création d'un mesh à la main

Objectif de l'atelier

Construire un maillage triangulaire en manipulant directement les **Vertex Buffers** et l'**Index Buffer** via l'objet `VertexData` de Babylon.js. Comprendre la structure intime d'un modèle 3D.

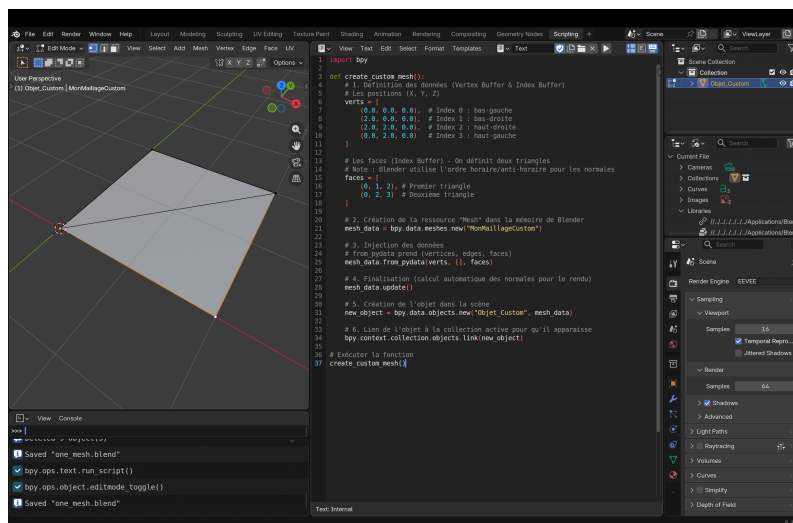


Fig. 3. – Le même maillage peut être créé visuellement dans Blender — les concepts de vertices, normales et UVs sont universels, indépendants du moteur utilisé.

VertexData : L'objet de construction

BABYLON.VertexData est le conteneur bas niveau pour les données géométriques. Il permet de définir manuellement chaque buffer avant de les appliquer à un mesh.

Buffers principaux :

- positions : Tableau de floats ($x_0, y_0, z_0, x_1, y_1, z_1, \dots$)
- normals : Tableau de floats ($nx_0, ny_0, nz_0, \dots$)
- uvs : Tableau de floats ($u_0, v_0, u_1, v_1, \dots$)
- indices : **Index Buffer** — tableau d'entiers désignant quels vertices forment chaque triangle.

Créer un triangle à la main.

```
// 1. Créer un mesh vide
const customMesh = new BABYLON.Mesh("custom", scene);

// 2. Définir les Vertex Buffers
const positions = [
    -1, -1, 0, // Vertex 0 : bas-gauche
    1, -1, 0, // Vertex 1 : bas-droite
    0, 1, 0 // Vertex 2 : haut-centre
];

const normals = [
    0, 0, 1, // Normale du Vertex 0 (vers la caméra)
    0, 0, 1, // Normale du Vertex 1
    0, 0, 1 // Normale du Vertex 2
];

const uvs = [
    0, 0, // UV du Vertex 0
    1, 0, // UV du Vertex 1
    0.5, 1 // UV du Vertex 2
];

// 3. Définir l'Index Buffer (quels vertices forment les triangles)
const indices = [0, 1, 2]; // Un seul triangle

// 4. Assembler le VertexData
const vertexData = new BABYLON.VertexData();
vertexData.positions = positions;
vertexData.normals = normals;
vertexData.uvs = uvs;
vertexData.indices = indices;

// 5. Appliquer au mesh
vertexData.applyToMesh(customMesh);

// 6. Assigner un matériau
const mat = new BABYLON.StandardMaterial("mat", scene);
mat.diffuseColor = new BABYLON.Color3(1, 0, 0);
mat.wireframe = false;
customMesh.material = mat;
```

L'Index Buffer : Pourquoi et Comment ?

L'**Index Buffer** (ou tableau d'indices) indique au GPU dans quel ordre lire les vertices pour former des triangles.

Pourquoi des indices ?

- Un vertex peut être partagé par plusieurs triangles (ex: un cube a 8 sommets mais 12 triangles).
- Sans index buffer, on dupliquerait les vertices partagés (gaspillage mémoire).
- Avec index buffer : 8 vertices + 36 indices au lieu de 36 vertices.

Convention de winding (sens d'enroulement) :

- Babylon.js utilise le **counter-clockwise** (sens inverse horaire) vu de l'extérieur.
- indices = [0, 1, 2] : le triangle est visible depuis +Z.
- indices = [0, 2, 1] : le triangle est visible depuis -Z (backface culling).

Démonstration : Construire un maillage de A à Z

Script complet pour le Babylon.js Playground. Ce code peut être collé directement dans le Playground. Il illustre la création d'un triangle et d'un carré (Quad) en manipulant les buffers.

```
export const createScene = function () {
    // 1. Initialisation de la scène basique
    const scene = new BABYLON.Scene(engine);

    // Caméra orbitale pour tourner autour de nos créations
    const camera = new BABYLON.ArcRotateCamera("camera", -Math.PI / 2, Math.PI / 2.5, 5,
BABYLON.Vector3.Zero(), scene);
    camera.attachControl(canvas, true);

    // Lumière basique
    const light = new BABYLON.HemisphericLight("light", new BABYLON.Vector3(0, 1, 0), scene);
    light.intensity = 0.7;

    // =====
    // EXEMPLE 1 : LE TRIANGLE
    // =====
    const triangleMesh = new BABYLON.Mesh("monTriangle", scene);

    const positionsTri = [
        -1, -1, 0,    // 0: bas-gauche
        1, -1, 0,     // 1: bas-droite
        0, 1, 0       // 2: haut-centre
    ];

    // Dans Babylon (Left-Handed), normales vers -1 en Z pour faire face à la caméra
    const normalsTri = [
        0, 0, -1,
        0, 0, -1,
        0, 0, -1
    ];

    const uvsTri = [
        0, 0,
        1, 0,
```



```

        0.5, 1
    ];

    const indicesTri = [0, 1, 2];

    const vertexDataTri = new BABYLON.VertexData();
    vertexDataTri.positions = positionsTri;
    vertexDataTri.normals = normalsTri;
    vertexDataTri.uvs = uvsTri;
    vertexDataTri.indices = indicesTri;

    vertexDataTri.applyToMesh(triangleMesh);

    // Matériau rouge pour le triangle
    const matTri = new BABYLON.StandardMaterial("matTri", scene);
    matTri.diffuseColor = new BABYLON.Color3(1, 0, 0);
    // matTri.wireframe = true; // Décommenter pour tester
    triangleMesh.material = matTri;

    triangleMesh.position.x = -1.5;

    // =====
    // EXEMPLE 2 : LE CARRÉ (QUAD) AVEC TEXTURE
    // =====
    const quadMesh = new BABYLON.Mesh("monCarre", scene);

    const positionsQuad = [
        -1, -1, 0, // 0 : bas-gauche
        1, -1, 0, // 1 : bas-droite
        1, 1, 0, // 2 : haut-droite
        -1, 1, 0 // 3 : haut-gauche
    ];

    const normalsQuad = [
        0, 0, -1,
        0, 0, -1,
        0, 0, -1,
        0, 0, -1
    ];

    const uvsQuad = [
        0, 0, // 0
        1, 0, // 1
        1, 1, // 2
        0, 1 // 3
    ];

    const indicesQuad = [0, 1, 2, 0, 2, 3];

    const vertexDataQuad = new BABYLON.VertexData();
    vertexDataQuad.positions = positionsQuad;
    vertexDataQuad.normals = normalsQuad;
    vertexDataQuad.uvs = uvsQuad;
    vertexDataQuad.indices = indicesQuad;

```

```

vertexDataQuad.applyToMesh(quadMesh);

// Matériau avec texture pour prouver que les UV fonctionnent
const matQuad = new BABYLON.StandardMaterial("matQuad", scene);
matQuad.diffuseTexture = new BABYLON.Texture("textures/checkerboard_basecolor.png",
scene);
quadMesh.material = matQuad;

quadMesh.position.x = 1.5;

// =====
// OUTILS PEDAGOGIQUES
// =====
// scene.debugLayer.show(); // Afficher l'inspecteur pour montrer les buffers

return scene;
};
    
```

Points interactifs à observer

1. **Le Backface Culling en direct :** Faites pivoter la caméra dans le Playground pour regarder derrière le triangle et le carré. Les objets deviendront invisibles. Cela illustre parfaitement que l'ordre des indices définit une face orientée d'un seul côté, optimisant ainsi le rendu GPU.
2. **Jouer avec le Wireframe :** Décommentez la ligne `matTri.wireframe = true;`. C'est un excellent moyen visuel de prouver que le carré (à droite) est bel et bien composé de deux triangles (la diagonale apparaît).
3. **Casser l'Index Buffer :** Changez `indicesQuad = [0, 1, 2, 0, 2, 3]` par `[0, 1, 2]`. Un seul triangle du carré s'affiche, prouvant le rôle des indices.
4. **Casser les UVs :** Changez la coordonnée UV du point 2 (haut-droite) de 1, 1 à 0.5, 0.5. La texture du damier se déformera instantanément, ce qui permet de constater le fonctionnement du « texture mapping ».

Session 2 : Transformations et Espaces de Coordonnées

🔗 Objectifs de la session

Comprendre le voyage d'un vertex à travers les espaces de coordonnées, maîtriser les matrices de transformation, et exploiter la hiérarchie de scène (parenting) pour animer des objets complexes.

Le voyage d'un Vertex : Le Pipeline de Transformation

Un vertex possède plusieurs positions

Un vertex ne possède pas « une » position, mais une série de positions selon le référentiel dans lequel on l'observe. À chaque étape, une matrice transforme les coordonnées d'un espace à l'autre.



Fig. 4. – Un vertex voyage d'un espace à l'autre jusqu'à ce qu'il devienne un pixel à l'écran.

Espace Local (Object Space)

L'origine est au centre de l'objet $(0, 0, 0)$. C'est l'espace dans lequel l'artiste modélise le mesh. Les coordonnées sont relatives à l'objet lui-même, indépendamment de sa position dans la scène.

Espace Monde (World Space)

La position de l'objet dans la scène globale. Tous les objets y sont exprimés dans un repère commun.

Transformation : Matrice Monde (Model Matrix) — combine la position, rotation et échelle de l'objet pour passer de l'espace local à l'espace monde.

Espace Caméra (View Space)

Le monde vu depuis l'œil du joueur. La caméra devient l'origine $(0, 0, 0)$, et les axes sont alignés sur son orientation.

Transformation : Matrice Vue (View Matrix) — transforme tout le monde pour qu'il soit vu depuis la caméra.

Espace Projection (Clip Space)

Conversion du volume 3D en un cube plat $[-1, 1]$. C'est ici qu'on définit la perspective : plus un objet est loin, plus il apparaît petit.

Transformation : Matrice de Projection — projette les coordonnées 3D en coordonnées 2D normalisées (NDC).

Le « Pourquoi » de la Matrice 4x4

Le problème

Une matrice 3x3 peut faire des rotations et des échelles, mais **pas de translations** (glissements). Or, on a besoin de déplacer les objets dans la scène.

La solution : Coordonnées Homogènes

On ajoute une 4ème dimension (w) aux coordonnées du vertex :

- Si $w = 1$, c'est un **Point** (déplaçable par translation).
- Si $w = 0$, c'est un **Vecteur** (direction pure, insensible à la translation).

Un point (x, y, z) devient $(x, y, z, 1)$ en coordonnées homogènes, permettant à une seule matrice 4x4 d'unifier rotations, échelles **et** translations.

❗ Concept clé

On multiplie par une matrice car cela permet de combiner 100 transformations complexes (rotation + scale + translation) en **une seule opération mathématique** sur le GPU. Le vertex shader n'effectue qu'une seule multiplication : $MVP = P \times V \times W$.

GPU

Graphics Processing Unit

10496 Cores

Lent - Tâches simples

CPU

Graphics Processing Unit

24 Cores

Rapide - Tâches complexes

Fig. 5. – Le GPU peut effectuer un grand nombre de multiplications de matrices.

Hiérarchie de scène : Le Parenting

Principe

Les objets 3D sont organisés en hiérarchie parent-enfant. La transformation d'un parent affecte tous ses enfants. C'est le **Scene Graph**, fondamental en programmation graphique.

En programmation, la matrice monde d'un enfant est le produit de la matrice monde du parent par sa matrice locale :

$$\text{MatriceMonde}_{\text{Enfant}} = \text{MatriceMonde}_{\text{Parent}} \times \text{MatriceLocale}_{\text{Enfant}}$$

Ce qui revient à additionner les positions :

$$\text{PosMonde}_{\text{Enfant}} = \text{PosMonde}_{\text{Parent}} + \text{PosLocale}_{\text{Enfant}}$$

💡 Visualisation dans Blender

Dans Blender, on peut visualiser cette hiérarchie avec un système solaire :

- Un **Empty** à l'origine du monde (0, 0, 0).
- Un objet **Parent** (ex: une Planète) à la position (3, 0, 0).
- Un objet **Enfant** (ex: une Lune) à la position (1, 0, 0) **relative au parent**.

Si on déplace le **Parent**, la Lune suit automatiquement — sa position locale n'a pas changé, mais sa position monde a été recalculée par multiplication des matrices.

Atelier : Manipuler la Hiérarchie dans Babylon.js

Les propriétés de base

- `.position` : Toujours **locale** par rapport au parent.
- `.rotation` (ou `.rotationQuaternion`) : Orientation locale.
- `.scaling` : Échelle locale (attention : si le parent est scalé, l'enfant l'est aussi !).

💡 Atelier : Le bras robotique

Plutôt qu'un système solaire, le bras robotique est excellent pour comprendre les matrices :

1. Créer une boîte Epaule.
2. Créer une boîte Bras, et faire `bras.parent = epaule`.
3. Décaler le bras en X de 2 unités.
4. **Expérience** : Faire tourner l'épaule sur l'axe Z.
 - **Observation** : Le bras tourne autour de l'épaule alors que sa propre rotation est à 0.
 - **Explication** : Babylon multiplie la matrice de l'épaule par celle du bras à chaque frame.

Code clé : *Hiérarchie et positions.*

```
// Création du bras robotique
const epaule = BABYLON.MeshBuilder.CreateBox("epaule", {size: 1}, scene);
epaule.position = new BABYLON.Vector3(0, 0, 0);

const bras = BABYLON.MeshBuilder.CreateBox("bras", {size: 0.5, height: 2}, scene);
bras.parent = epaule;
bras.position = new BABYLON.Vector3(2, 0, 0); // Local : 2 unités à droite de l'épaule

// Faire tourner l'épaule : le bras tourne autour d'elle
scene.registerBeforeRender(() => {
  epaule.rotation.z += 0.01;
});

// Position locale (par rapport au parent)
lune.position = new BABYLON.Vector3(2, 0, 0);

// Récupérer la position réelle dans le monde (après multiplication des matrices)
let worldPos = lune.getAbsolutePosition();
```

```
// Forcer la mise à jour des matrices si on en a besoin immédiatement
lune.computeWorldMatrix(true);
```

Concepts clés à retenir

L'ordre des multiplications importe

Faire une Rotation **puis** une Translation n'est pas la même chose qu'une Translation **puis** une Rotation :

- Rotation puis Translation : l'objet tourne sur lui-même, puis est déplacé.
- Translation puis Rotation : l'objet est déplacé, puis tourne autour de l'origine (effet pivot).

C'est pourquoi l'ordre des opérations dans la composition de matrices est crucial.

Le Scale hérité

On évite de mettre une échelle sur un objet Parent : cela déforme les enfants de manière souvent indésirable. Si un parent a un scale de 2, tous ses enfants voient leurs dimensions doublées, ce qui peut causer des bugs visuels et des problèmes de physique.

Transformation de vecteurs vs points

Les normales sont des **vecteurs** ($w = 0$), pas des points ($w = 1$). Elles ne subissent donc **pas** la translation de la matrice — seulement la rotation. C'est pourquoi une normale pointe toujours dans la même direction, même si on déplace l'objet.

💡 Question de réflexion

« Si je déplace ma caméra de 5 unités vers la droite, que doit faire le GPU avec tous les objets du monde pour qu'ils apparaissent correctement à l'écran ? »

Réponse : Il doit tous les déplacer de 5 unités vers la **gauche**. C'est le rôle de la Matrice Vue — la caméra est fixe, c'est le monde qui bouge autour d'elle.

Session 3 : Création de Shaders dans Babylon.js (Partie 1)

🔗 Objectifs de la session

Comprendre le pipeline de rendu programmable et écrire ses premiers shaders GLSL dans Babylon.js.

Qu'est-ce qu'un Shader ?

Définition

Un shader est un programme exécuté sur la carte graphique (GPU). Il contrôle comment les sommets sont transformés (Vertex Shader) et comment les pixels sont colorés (Fragment/Pixel Shader).

Le Pipeline de Rendu Programmable

Étapes du pipeline

1. **Vertex Shader** : Transforme chaque sommet de l'Object Space au Clip Space.
2. **Rasterization** : Le GPU convertit les triangles en pixels.
3. **Fragment Shader** : Calcule la couleur de chaque pixel.
4. **Output Merger** : La couleur finale est écrite dans le framebuffer.

GLSL : Le langage des Shaders

Syntaxe GLSL de base

```
// Vertex Shader
attribute vec3 position;
uniform mat4 worldViewProjection;

void main() {
    gl_Position = worldViewProjection * vec4(position, 1.0);
}

// Fragment Shader
void main() {
    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0); // Rouge
}
```

Shaders dans Babylon.js : ShaderMaterial

Créer un ShaderMaterial.

```
BABYLON.Effect.ShadersStore["customVertexShader"] = `
    attribute vec3 position;
    uniform mat4 worldViewProjection;
    varying vec3 vPosition;

    void main() {
```

```

        vPosition = position;
        gl_Position = worldViewProjection * vec4(position, 1.0);
    }
`;

BABYLON.Effect.ShadersStore["customFragmentShader"] = `
    varying vec3 vPosition;

    void main() {
        gl_FragColor = vec4(vPosition.x, vPosition.y, vPosition.z, 1.0);
    }
`;

const shaderMaterial = new BABYLON.ShaderMaterial(
    "shader", scene, { vertex: "custom", fragment: "custom" },
    { attributes: ["position"], uniforms: ["worldViewProjection"] }
);
mesh.material = shaderMaterial;

```

Uniforms et Varyings

Communication entre shaders

- **Uniforms** : Variables passées du CPU (JavaScript) au GPU (shader). Constantes pour tous les sommets/pixels.
- **Attributes** : Données par sommet (position, normal, UV).
- **Varyings** : Variables passées du Vertex Shader au Fragment Shader (interpolées par pixel).

Session 4 : Création de Shaders dans Babylon.js (Partie 2)

🔗 Objectifs de la session

Approfondir les shaders : éclairage custom, textures dans les shaders, et effets visuels.

Éclairage dans un Shader

Modèle d'éclairage Lambertien (Diffuse)

```
uniform vec3 lightDirection;
uniform vec3 lightColor;
varying vec3 vNormal;
varying vec3 vColor;

void main() {
    float intensity = max(dot(normalize(vNormal), normalize(lightDirection)), 0.0);
    gl_FragColor = vec4(vColor * lightColor * intensity, 1.0);
}
```

Le produit scalaire entre la normale de la surface et la direction de la lumière donne l'intensité de l'éclairage diffus.

Textures dans les Shaders

Sampling de texture

```
uniform sampler2D diffuseTexture;
varying vec2 vUV;

void main() {
    vec4 texColor = texture2D(diffuseTexture, vUV);
    gl_FragColor = texColor;
}
```

En Babylon.js, on passe la texture via le ShaderMaterial :

```
shaderMaterial.setTexture("diffuseTexture", myTexture);
```

Normal Mapping

Normal Map

Une normal map est une texture RGB où chaque texel encode une normale ($R = x, G = y, B = z$). Elle permet de simuler des détails géométriques sans ajouter de polygones.

Dans le shader, on remplace la normale de la géométrie par la normale lue dans la texture (après conversion de l'espace tangent à l'espace monde).

Effets visuels : Procedural Shaders

Shader procédural : damier.

```
varying vec2 vUV;  
  
void main() {  
    float checker = mod(floor(vUV.x * 10.0) + floor(vUV.y * 10.0), 2.0);  
    vec3 color = mix(vec3(1.0), vec3(0.0), checker);  
    gl_FragColor = vec4(color, 1.0);  
}
```

Node Material Editor

Node Material

Babylon.js propose un éditeur visuel de shaders (Node Material Editor) permettant de créer des shaders sans écrire de GLSL, en connectant des nœuds (comme dans Blender ou Unreal Engine). C'est un excellent outil pour prototyper rapidement des effets.

Session 5 : Création d'éclairage dans Babylon.js

🔗 Objectifs de la session

Maîtriser les différents types de lumières, leurs paramètres, et comment elles interagissent avec les matériaux.

Types de Lumières dans Babylon.js

Lumières intégrées

- **HemisphericLight** : Lumière ambiante simulée (ciel + sol).
- **DirectionalLight** : Lumière directionnelle (ex: soleil). Tous les rayons sont parallèles.
- **PointLight** : Lumière ponctuelle omnidirectionnelle (ex: ampoule).
- **SpotLight** : Lumière conique (ex: lampe torche). Possède un angle et un penumbra.

Paramètres de Lumière

Propriétés communes

- `light.diffuse` : Couleur de la lumière (Color3).
- `light.specular` : Couleur du reflet spéculaire.
- `light.intensity` : Intensité (multiplicateur).
- `light.range` : Portée de la lumière (pour Point/Spot).
- `light.direction` : Direction (pour Directional/Spot).

Modèles d'Éclairage

Modèles physiques

- **Lambert (Diffuse)** : $k_d = \max(N \cdot L, 0)$ — lumière répartie uniformément.
- **Phong (Spéculaire)** : $k_s = \max(R \cdot V, 0)^{\text{shininess}}$ — reflet brillant.
- **Blinn-Phong** : Variante utilisant le vecteur half-angle $H = \frac{L+V}{|L+V|}$.
- **PBR (Physically Based Rendering)** : Modèle réaliste basé sur la conservation d'énergie. Utilisé par le `PBRMaterial` de Babylon.js.

PBRMaterial dans Babylon.js

Propriétés PBR

- `albedoColor` : Couleur de base (équivalent diffuse mais en PBR).
- `metallic` : 0 = diélectrique (bois, plastique), 1 = métal.
- `roughness` : 0 = surface lisse (miroir), 1 = surface rugueuse (caoutchouc).
- `environmentTexture` : Image HDR pour les réflexions et l'éclairage ambiant (IBL).

Multiples Lumières et Performance

Limite de lumières

Chaque lumière ajoutée augmente le coût du shader. Babylon.js limite le nombre de lumières simultanées affectant un mesh (par défaut 4). Au-delà, il faut utiliser des techniques avancées (light pre-pass, deferred rendering).

Session 6 : Stratégies d'Ombres et Réflexions

🔗 Objectifs de la session

Comprendre les techniques d'ombres temps réel et de réflexions, et les implémenter dans Babylon.js.

Shadow Mapping

Principe

L'ombre est calculée en rendant la scène depuis le point de vue de la lumière. On stocke les profondeurs dans une **Shadow Map** (texture de profondeur). Au rendu final, on compare la profondeur du fragment à celle stockée dans la shadow map : si le fragment est plus loin, il est dans l'ombre.

Ombres dans Babylon.js

ShadowGenerator.

```
const shadowGenerator = new BABYLON.ShadowGenerator(1024, light);
shadowGenerator.addShadowCaster(mesh);
shadowGenerator.useExponentialShadowMap = true;
```

```
// Le sol reçoit l'ombre
ground.receiveShadows = true;
```

Types de Shadow Maps

Variantes

- **Basic Shadow Map** : Simple, qualité limitée (aliasing).
- **Exponential Shadow Map (ESM)** : Lisse les bords d'ombre.
- **Poisson Sampling** : Flou stochastique pour adoucir les bords.
- **Variance Shadow Map (VSM)** : Permet le flou gaussien sur la shadow map.
- **Cube Shadow Map** : Pour les PointLight (6 faces).

Réflexions

Techniques de réflexion

- **Environment Mapping** : Une texture cubique (cube map) représente l'environnement. Réflexion via `reflectionTexture` sur le matériau.
- **Planar Reflections** : Rendu de la scène depuis un miroir. Coûteux mais précis.
- **Screen Space Reflections (SSR)** : Réflexions calculées dans l'espace écran en réutilisant le framebuffer. Approximation, ne reflète que ce qui est visible à l'écran.

Réflexions dans Babylon.js

CubeMap réflexion.

```
const reflectionTexture = new BABYLON.CubeTexture(  
    "textures/environment.env", scene  
);  
material.reflectionTexture = reflectionTexture;  
material.reflectionFresnelParameters = new BABYLON.FresnelParameters();  
material.reflectionFresnelParameters.bias = 0.1;
```

Image-Based Lighting (IBL)

IBL

Technique PBR où l'environnement (cube map HDR) sert à la fois de source de réflexions et d'éclairage ambiant. C'est le standard moderne pour un rendu réaliste. Babylon.js supporte l'IBL via `environmentTexture` sur la scène.

Session 7 : Création de Post-Process dans Babylon.js

🔗 Objectifs de la session

Apprendre à appliquer des effets visuels après le rendu de la scène (post-processing) : flou, bloom, correction de couleur, etc.

Qu'est-ce qu'un Post-Process ?

Définition

Un post-process est un effet appliqué sur l'image finale rendue (framebuffer) avant son affichage. Il s'exécute dans un shader plein écran (fullscreen quad) qui lit la texture de rendu et applique une transformation par pixel.

Post-Process Pipeline dans Babylon.js

Pipeline de post-process.

```
const pipeline = new BABYLON.DefaultRenderingPipeline(
  "default", true, scene, [camera]
);
pipeline.bloomEnabled = true;
pipeline.bloomThreshold = 0.8;
pipeline.bloomWeight = 0.3;

pipeline.imageProcessing.contrast = 1.2;
pipeline.imageProcessing.exposure = 1.0;
pipeline.imageProcessing.vignetteEnabled = true;
```

Post-Process Intégrés

Effets disponibles

- **Bloom** : Halos lumineux autour des zones brillantes.
- **Depth of Field (DoF)** : Flou de profondeur (arrière-plan flou).
- **Vignette** : Assombrissement des bords de l'image.
- **Chromatic Aberration** : Décalage des canaux RGB (effet lentille).
- **Grain** : Bruit filmique.
- **Fxaa** : Anti-aliasing rapide.
- **Screen Space Reflections (SSR)** : Réflections temps réel.

Post-Process Custom

Shader personnalisé en post-process.

```
const postProcess = new BABYLON.PostProcess(
  "custom", "customEffect", ["time"], null, 1.0, camera
);
```

```
BABYLON.Effect.ShadersStore["customEffectFragmentShader"] = `
  varying vec2 vUV;
```

```

uniform sampler2D textureSampler;
uniform float time;

void main() {
    vec2 uv = vUV;
    uv.x += sin(uv.y * 10.0 + time) * 0.01;
    gl_FragColor = texture2D(textureSampler, uv);
}
`;

postProcess.onApply = (effect) => {
    effect.setFloat("time", performance.now() / 1000);
};

```

HDR et Tone Mapping

Tone Mapping

Le rendu HDR (High Dynamic Range) calcule la lumière avec des valeurs supérieures à 1.0. Le tone mapping ramène ces valeurs dans la plage $[0, 1]$ pour l’affichage. Babylon.js propose plusieurs opérateurs : ACES, Reinhard, etc.

Session 8 : Rendering Pipeline (Partie 1)

🔗 Objectifs de la session

Comprendre l'architecture du pipeline de rendu : du modèle 3D au pixel à l'écran.

Vue d'Ensemble du Pipeline

Étapes du pipeline graphique

1. **Application Stage (CPU)** : Le code (JS) prépare les données : matrices, paramètres, culling.
2. **Geometry Processing (GPU)** : Vertex Shader, Tessellation, Geometry Shader.
3. **Rasterization (GPU)** : Conversion des triangles en fragments (pixels).
4. **Pixel Processing (GPU)** : Fragment Shader, tests de profondeur, blending.
5. **Output Merger** : Écriture dans le framebuffer.

Vertex Shader en Détail

Responsabilités

Le Vertex Shader transforme chaque sommet :

- Object Space → World Space (matrice **world**)
- World Space → View Space (matrice **view**)
- View Space → Clip Space (matrice **projection**)
- $$MVP = P \times V \times W$$

Il peut aussi passer des données au Fragment Shader (normales, UVs, couleurs).

Rasterization

Du triangle au pixel

Le rasterizer détermine quels pixels sont couverts par chaque triangle. Il interpole les varyings (UV, normales, couleurs) à travers la surface du triangle.

Fragment Shader en Détail

Responsabilités

Le Fragment Shader calcule la couleur finale de chaque pixel :

- Sampling de textures
- Calculs d'éclairage
- Effets procéduraux

Il ne connaît rien de la géométrie — seulement des varyings interpolés.

Tests et Blending

Tests du framebuffer

- **Depth Test** : Compare la profondeur du fragment avec le Z-buffer. Si le fragment est derrière, il est ignoré.
- **Stencil Test** : Masque basé sur un buffer de stencil (ex: silhouettes, reflets).
- **Blending** : Mélange la couleur du fragment avec celle du framebuffer (transparence).

Session 9 : Rendering Pipeline (Partie 2)

🔗 Objectifs de la session

Approfondir le pipeline : techniques avancées de rendu, deferred rendering, et architecture moderne.

Forward vs Deferred Rendering

Forward Rendering

Chaque objet est rendu un par un. Pour chaque fragment, on calcule l'éclairage de toutes les lumières.

- **Avantage** : Simple, MSAA compatible, bon pour peu de lumières.
- **Inconvénient** : Coût $O(\text{objets} \times \text{lumières})$, overdraw (pixels rendus plusieurs fois).

Deferred Rendering

On rend d'abord la géométrie dans des G-Buffers (positions, normales, couleurs, profondeur). L'éclairage est calculé dans un second pass en plein écran.

- **Avantage** : Coût d'éclairage indépendant du nombre d'objets, $O(\text{pixels} \times \text{lumières})$.
- **Inconvénient** : Gourmand en mémoire (G-Buffers), MSAA difficile, transparence problématique.

G-Buffer

Composition typique

- **RTo** : Albedo (RGB) + AO (A)
- **RT1** : Normales (RGB) + Roughness (A)
- **RT2** : Metallic (R) + Specular (G) + Depth (B)

Chaque Render Target stocke des informations géométriques et matérielles.

Culling

Optimisations de géométrie

- **Frustum Culling** : On ignore les objets hors du champ de vision de la caméra.
- **Occlusion Culling** : On ignore les objets cachés derrière d'autres.
- **Backface Culling** : On ignore les faces arrière des triangles (normale pointant away).

Draw Calls et Batching

💡 Le goulot d'étranglement

Chaque **draw call** (appel de rendu) a un coût CPU. Réduire le nombre de draw calls est crucial :

- **Instancing** : Rendre N copies d'un même mesh en un seul draw call.

- **Merging** : Fusionner les meshes statiques.
- **Texture Atlas** : Regrouper les textures pour éviter de changer de matériau.

Babylon.js et le Pipeline

Rendering Pipeline de Babylon.js

Babylon.js utilise par défaut le Forward Rendering. Le Deferred Rendering est disponible via le plugin `@babylonjs/materials`. La scène gère automatiquement le frustum culling, le tri des objets transparents, et le batching des lumières.

Session 10 : Techniques d'Optimisation et Profilage (Partie 1)

🔗 Objectifs de la session

Apprendre à mesurer et optimiser les performances d'une application 3D temps réel.

Pourquoi Optimiser ?

Le budget de performance

Pour maintenir 60 FPS, chaque frame doit être rendue en moins de 16.6ms. Le rendu graphique n'est qu'une partie de ce budget (avec la logique de jeu, l'audio, le réseau).

Outils de Profilage

Outils disponibles

- **Babylon.js Inspector** : `scene.debugLayer.show()` — statistiques de scène, draw calls, FPS, mémoire GPU.
- **Chrome DevTools** : Performance tab, GPU tab, Memory tab.
- **WebGL Inspector / Spector.js** : Capture des draw calls WebGL, inspection des buffers et shaders.

Métriques Clés

Indicateurs de performance

- **FPS** : Images par seconde (cible: 60).
- **Draw Calls** : Nombre d'appels de rendu par frame (cible: < 1000).
- **Triangles** : Nombre total de triangles rendus.
- **Texture Memory** : Mémoire GPU utilisée par les textures.
- **Shader Complexity** : Nombre d'instructions dans les shaders.

Optimisation Géométrique

Réduction de la complexité

- **LOD (Level of Detail)** : Plusieurs versions d'un mesh avec des niveaux de détail décroissants. On choisit le LOD selon la distance à la caméra.
- **Mesh Simplification** : Réduction du nombre de polygones.
- **Frustum Culling** : Déjà géré par Babylon.js, mais peut être amélioré avec des **occluders**.

Optimisation des Draw Calls

Techniques de batching

- **Thin Instances** : Babylon.js permet de rendre des milliers d'instances d'un mesh en un seul draw call.

```
mesh.thinInstanceCount = 1000;  
mesh.thinInstanceSetBuffer("matrix", matricesBuffer, 16);
```

- **Merge Meshes** : BABYLON.Mesh.MergeMeshes(meshes) fusionne plusieurs meshes statiques en un seul.

Session 11 : Techniques d'Optimisation et Profilage (Partie 2)

🔗 Objectifs de la session

Optimisation des textures, des shaders, et techniques avancées de performance GPU.

Optimisation des Textures

Stratégies

- **Texture Compression** : Formats compressés GPU (BasisU, KTX2) qui réduisent la mémoire et la bande passante.
- **Mipmapping** : Versions pré-calculées de la texture à différentes résolutions. Le GPU choisit la mip level selon la distance.
- **Texture Atlas** : Regrouper plusieurs textures en une seule pour réduire les changements de matériau.
- **Resolution** : Adapter la résolution des textures à l'importance visuelle de l'objet.

Optimisation des Shaders

Bonnes pratiques GLSL

- Éviter les branches conditionnelles (`if`) dans les shaders — préférer `mix`, `step`.
- Minimiser les calculs dans le Fragment Shader (il s'exécute millions de fois par frame).
- Précalculer ce qui peut l'être dans le Vertex Shader.
- Utiliser les built-in GLSL (plus optimisés que les fonctions custom).

Level of Detail (LOD) dans Babylon.js

LOD automatique.

```
mesh.addLODLevel(20, highDetailMesh);  
mesh.addLODLevel(50, mediumDetailMesh);  
mesh.addLODLevel(100, lowDetailMesh);  
mesh.addLODLevel(200, null); // Rien rendu au-delà
```

Gestion de la Mémoire GPU

💡 Règles d'or

- Libérer les ressources : `mesh.dispose()`, `texture.dispose()`, `material.dispose()`.
- Réutiliser les géométries (un seul mesh, multiples instances).
- Surveiller les fuites : le Babylon.js Inspector affiche la mémoire GPU.

WebGPU vs WebGL

Le futur du rendu web

Babylon.js supporte WebGPU, la nouvelle API graphique du web. WebGPU offre :

- Accès plus direct au GPU (moins d'overhead CPU).
- Compute Shaders (calculs généraux sur GPU).
- Meilleure performance pour les scènes complexes.

On bascule entre WebGL et WebGPU via :

```
const engine = new BABYLON.WebGPUEngine(canvas);  
await engine.initAsync();
```


Session 12 : Exploration d'une API de Programmation Graphique

🔗 Objectifs de la session

Plonger dans une API graphique bas niveau (WebGPU) pour comprendre ce qui se cache sous l'abstraction de Babylon.js.

APIs Graphiques : Vue d'Ensemble

Niveaux d'abstraction

- **Haut niveau** : Babylon.js, Three.js — moteur complet, scène, matériaux, éclairage.
- **Moyen niveau** : WebGL, WebGPU — accès direct au pipeline GPU, mais gestion manuelle.
- **Bas niveau** : OpenGL, Vulkan, DirectX 12, Metal — APIs natives, contrôle total.

WebGPU : La Nouvelle API Web

Concepts fondamentaux

- **Adapter** : Représente le GPU physique.
- **Device** : La connexion logique au GPU.
- **Pipeline** : Décrit le rendu (shaders, format de sortie, blend mode).
- **Buffers** : Données envoyées au GPU (vertex, index, uniform).
- **Textures** : Images stockées sur le GPU.
- **Command Encoder** : Enregistre les commandes de rendu, soumis au GPU en un seul **submit**.

Comparaison WebGL vs WebGPU

Différences clés

Aspect	WebGL	WebGPU
Shader Language	GLSL	WGSL
Paradigme	État global	Objets explicites
Compute	Non	Oui (Compute Shaders)
Performance	Bonne	Meilleure (moins d'overhead)
Maturité	Très mature	Récent, en évolution

WebGPU dans Babylon.js

Engine WebGPU.

```
const engine = new BABYLON.WebGPUEngine(canvas);
await engine.initAsync();
```

```
const scene = new BABYLON.Scene(engine);
// ... même code que WebGL
```

```
engine.runRenderLoop(() => scene.render());
```

Babylon.js abstrait la différence entre WebGL et WebGPU. Le même code de scène fonctionne sur les deux.

Pour les compute shaders :

```
const computeShader = new BABYLON.ComputeShader(
    "compute", engine, { computeSource: wgsLCode },
    [{ type: "storage", name: "output", buffer: storageBuffer }]
);
computeShader.dispatch(64, 1, 1);
```

WGSL : Le Langage de Shader WebGPU

Exemple WGSL

```
@vertex
fn vs_main(@location(0) position: vec3<f32>) -> @builtin(position) vec4<f32> {
    return vec4<f32>(position, 1.0);
}

@fragment
fn fs_main() -> @location(0) vec4<f32> {
    return vec4<f32>(1.0, 0.0, 0.0, 1.0);
}
```

WGSL est plus typé et structuré que GLSL, conçu pour la sécurité et la performance.

Conclusion du Cours

💡 Récapitulatif

Au cours de cette session, nous avons exploré l'API sous-jacente qui alimente le rendu moderne sur le web. Comprendre WebGPU permet de mieux utiliser Babylon.js et d'optimiser les performances au maximum.